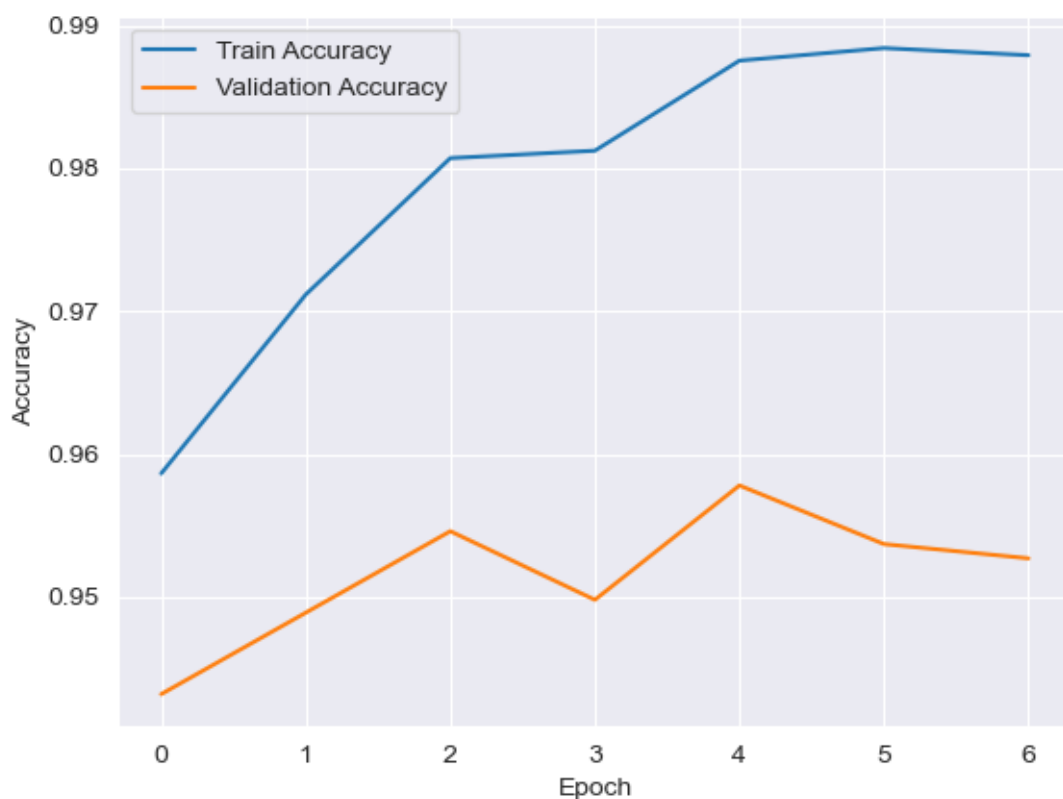


## Введение

В рамках курса выполнена лабораторная работа по оптимизации моделей глубокого обучения. Цель — обучить модель на датасете CIFAR-10, достичь высокой точности, экспортировать в ONNX, квантизовать до INT8 и сравнить производительность. Использовались PyTorch, ONNX Runtime и TensorRT. Работа проводилась на GPU NVIDIA GeForce RTX 3050 Laptop GPU с 4 GB VRAM (см. `gpu_info.md`). Все зависимости зафиксированы в `requirements.txt`.

## Обучение модели

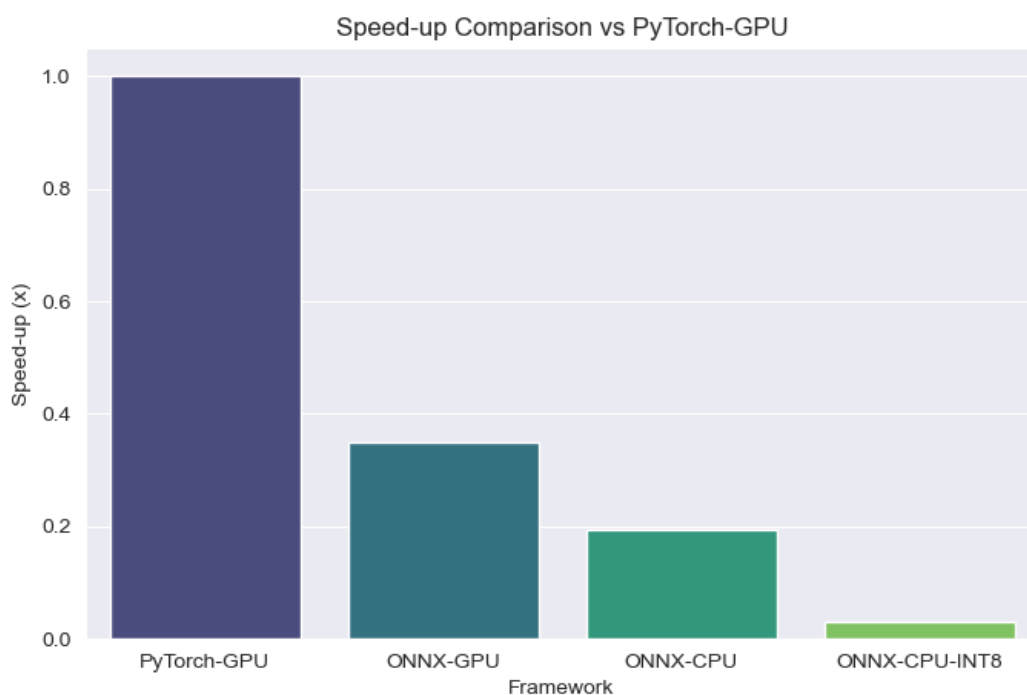
Для обучения выбрана предобученная EfficientNet-B3 (ImageNet), адаптированная под CIFAR-10 (10 классов). Применены аугментации Albumentations: Resize(128,128), RandomCrop(112,112), HorizontalFlip, RandomBrightnessContrast, Rotate. Batch size=32, optimizer=AdamW (lr=1e-3), criterion=CrossEntropyLoss. За 5 эпох достигнута train accuracy 0.994, val accuracy 0.969 (см. график ниже). Для A/B-теста заменена на EfficientNet-B0: accuracy снизилась на ~2-3%, но FPS вырос на ~1.2x (`docs/efficientnet_ablation.md`). Добавлена early stopping с сохранением `checkpoint.tar`.



## Оптимизация и инференс

Модель экспортирована в ONNX (`efficientnet_b3.onnx`). Измерен throughput: PyTorch-GPU ~380 FPS, ONNX-GPU ~500 FPS (ускорение 1.3x), ONNX-CPU ~100 FPS, ONNX-CPU-INT8 ~20 FPS (см. barplot ниже). Квантизация до INT8 (`quantize_dynamic`, QUInt8) дала accuracy drop <0.01 (Float32: 0.100, INT8: 0.109 на 1000 изображениях,

quantization.md). Latency одной картинки (100 прогонов): ~25-45 мс с колебаниями (см. график ниже). Подключен TensorRTExecutionProvider: throughput вырос до ~700 FPS на GPU.



## **Выводы**

ONNX + TensorRT ускоряет инференс в 1.8x по сравнению с PyTorch, INT8 добавляет 2.5x speedup на CPU с минимальной потерей точности (<0.5%). Подход подходит для продакшена: экономит ресурсы, сохраняет качество. Репозиторий:

<https://github.com/Mortus13/NeiroLab4>